Practical 8

Implementation of graph traversal using BFS and DFS

1. Introduction

Graph traversal is the process of visiting every vertex of a graph exactly once in a systematic way. It is a fundamental operation used in searching, path finding, connectivity checking and many other graph algorithms.

Breadth First Search (BFS) visits the vertices level by level, starting from the source vertex, and uses a queue. Depth First Search (DFS) goes as deep as possible along each branch before backtracking, and uses recursion (the call stack). Both algorithms use a visited array to make sure that no vertex is processed more than once.

2. Aim

To implement graph traversal using Breadth First Search (BFS) and Depth First Search (DFS) in C++ on a graph represented using an adjacency list, and to display the order in which the vertices are visited.

3. Objectives

- To understand the concept of graph traversal.
- To understand the BFS approach using a queue.
- To understand the DFS approach using recursion.
- To represent a graph using an adjacency list.
- To compare the traversal order of BFS and DFS.
- To analyze the time and space complexity of both algorithms.
- To implement both algorithms using C++.

4. Algorithm

4.1 BFS Algorithm

Step 1: Start.

Step 2: Read the graph as an adjacency list and the starting vertex.

Step 3: Create a visited array initialized to false and an empty queue.

Step 4: Mark the starting vertex as visited and insert it into the queue.

Step 5: While the queue is not empty, remove the front vertex and print it.

Step 6: For each adjacent vertex that is not visited, mark it as visited and insert it into the queue.

Step 7: Repeat Steps 5 and 6 until the queue becomes empty.

Step 8: Stop.

4.2 DFS Algorithm

Step 1: Start.

Step 2: Read the graph as an adjacency list and the starting vertex.

Step 3: Create a visited array initialized to false.

Step 4: Mark the current vertex as visited and print it.

Step 5: For each adjacent vertex that is not visited, recursively call DFS on it.

Step 6: When all adjacent vertices are processed, return (backtrack).

Step 7: Stop.

5. Pseudocode

5.1 BFS

```
BEGIN
FUNCTION BFS(start, adj, visited)
    Create an empty queue Q
    visited[start] ← true
    ENQUEUE(Q, start)
    WHILE Q is not empty
        node ← DEQUEUE(Q)
        PRINT node
        FOR EACH i IN adj[node]
            IF visited[i] = false THEN
                visited[i] ← true
                ENQUEUE(Q, i)
            END IF
        END FOR
    END WHILE
END FUNCTION
END
```

5.2 DFS

```
BEGIN
FUNCTION DFS(node, adj, visited)
    visited[node] ← true
    PRINT node
    FOR EACH i IN adj[node]
        IF visited[i] = false THEN
            DFS(i, adj, visited)
        END IF
    END FOR
END FUNCTION
END
```

Code:

BFS Code

```
#include <iostream>
#include <vector>
#include <queue>
```

```
using namespace std;

void BFS(int start, vector<int> adj[], bool visited[]) {
    queue<int> q;

    visited[start] = true;
    q.push(start);

    while (!q.empty()) {
        int node = q.front();
        q.pop();

        cout << node << " ";

        for (int i : adj[node]) {
            if (!visited[i]) {
                visited[i] = true;
                q.push(i);
            }
        }
    }
}

int main() {
    int vertices = 5;

    vector<int> adj[5];
    bool visited[5] = {false};

    adj[0].push_back(1);
    adj[0].push_back(2);
    adj[1].push_back(3);
    adj[1].push_back(4);

    cout << "BFS Traversal: ";
    BFS(0, adj, visited);

    return 0;
}
```

Output:

```
BFS Traversal: 0 1 2 3 4
```

DFS Code

```
#include <iostream>
#include <vector>
using namespace std;

void DFS(int node, vector<int> adj[], bool visited[]) {
    visited[node] = true;
    cout << node << " ";

    for (int i : adj[node]) {
        if (!visited[i]) {
```

```
            DFS(i, adj, visited);
        }
    }
}

int main() {
    int vertices = 5;

    vector<int> adj[5];
    bool visited[5] = {false};

    adj[0].push_back(1);
    adj[0].push_back(2);
    adj[1].push_back(3);
    adj[1].push_back(4);

    cout << "DFS Traversal: ";
    DFS(0, adj, visited);

    return 0;
}
```

Output:

```
DFS Traversal: 0 1 3 4 2
```

The graph used has 5 vertices (0 to 4) with the edges 0→1, 0→2, 1→3 and 1→4. BFS starting from vertex 0 visits the vertices level by level (0, then 1 and 2, then 3 and 4), while DFS goes deep first along 0 → 1 → 3, backtracks to visit 4, and finally visits 2.

6. Advantages

BFS

- Finds the shortest path (minimum number of edges) in an unweighted graph.
- Visits vertices level by level, which is useful for level order problems.
- Guaranteed to find a solution if one exists.

DFS

- Requires less memory than BFS for deep, narrow graphs.
- Simple to implement using recursion.
- Useful for cycle detection, topological sorting and finding connected components.

7. Disadvantages

BFS

- Requires more memory because the queue can hold many vertices at once.
- Can be slow for graphs with a very large branching factor.

DFS

- Does not guarantee the shortest path.
- Recursion can cause stack overflow for very deep graphs.
- May go deep into a branch that does not contain the solution.

8. Complexity Analysis

Complexity	**BFS**	**DFS**
Time Complexity	O(V + E)	O(V + E)
Space Complexity	O(V)	O(V)

Where:

- V = number of vertices
- E = number of edges

9. Conclusion

Graph traversal using BFS and DFS was successfully implemented in C++. BFS uses a queue and visits the vertices level by level, whereas DFS uses recursion and explores each branch as deep as possible before backtracking. Both algorithms visit every vertex once and have a time complexity of O(V + E). Thus, BFS and DFS provide efficient and systematic approaches for traversing a graph, and the choice between them depends on the problem being solved.